

COMP | IT | AIDS

MICRO PROCESSOR

SPPU ENDSEM

UNIT 4

ONE SHOT 

PROTECTION



+PYQ'S COVERED

+FREE HANDWRITTEN NOTES

SUBSCRIBE



UNIT - 04 PROTECTION

* Protection Mechanism in 80386 :-

These mechanism includes some operations which verifies the data and system protection.

Need of Protection :

- It detects and identify bugs
- 1) 80386 supports sophisticated applications with thousands of program modules.
- 2) In such applications, finding a bug is very difficult. Also if identified a bug how to eliminate it as fast as possible and how their damage can be tightly confined; is a big challenge.
- 3) To help debug applications faster & make them more robust in production 80386 contains mechanisms to verify memory accesses and instruction execution for conformance to protection criteria.
- 4) These mechanism may be used or ignored, according to system objectives.

Aspects of protection Mechanism :

- 1) Type checking
- 2) Limit checking
- 3) Restriction of Addressable domain
- 4) Restriction of procedure entry points
- 5) Restriction of instruction set.

protection = integral part of hardware memory management H/w

applies to $\left\{ \begin{array}{l} \text{segmentation} \\ \text{paging} \end{array} \right\}$ translation

each reference of memory is checked by H/w to verify protection criteria.

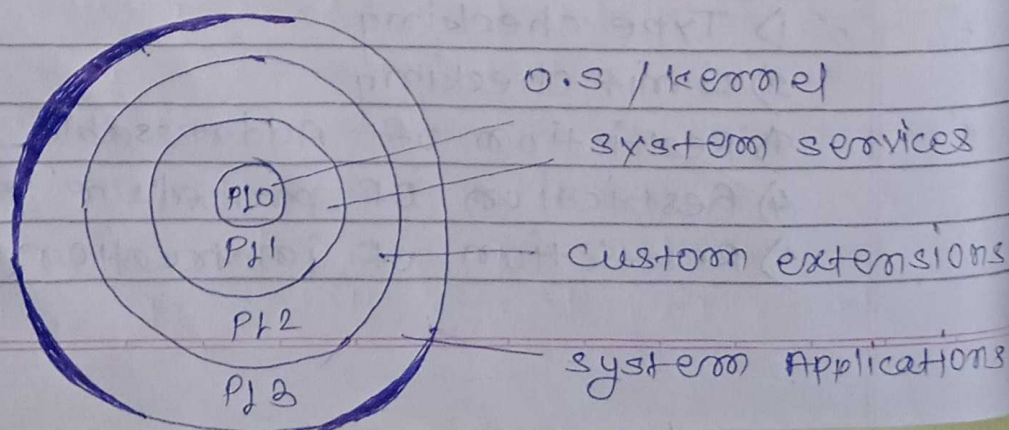
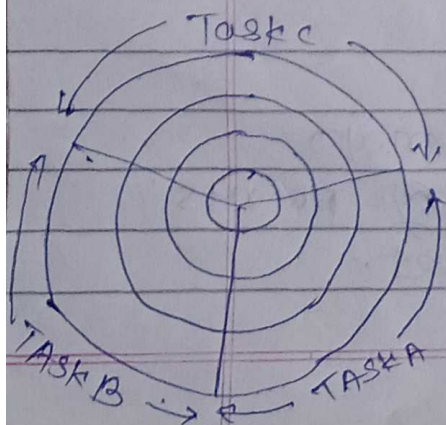
All checks are before starting memory cycle. Any violation that cycle from starting & results an exception.

No performance $\xrightarrow{\text{Reason}}$ checks are performed concurrently with address formation.
penalty

exception $\rightarrow$ invalid attempt to access memory
privilege $\rightarrow$ central to several aspects of protection

$\rightarrow$ degree to which procedure can be trusted not to make might that affect data or procedures.
 $\rightarrow$ degree of protection that data structure should have from less trusted procedures.

Note: both diagram should draw together



PLO - PL3 → Privilege levels

not necessary to use all PL's

* Privilege level

It is given protection levels from PLO to PL3.

1) Current Privilege level (CPL)

PL level at which task is currently executing which equals PL of code segment being executed. It can also be determined by examining lowest 2 bits of CS register, except confirming code segments.

CRAFTERS 

2) Descriptor privilege level (DPL)

Least privilege PL at which task may access that descriptor.

It is determined by bits 6:5 in Access Right Byte of descriptor.

It is associated with segment descriptor in GDT or LDT. It defines minimum CPL required to access segment or execute call gate to that segment.

3) I/O Privilege level (IOPL)

It is a 2 bit field in EFLAGS register that control access to I/O ports via IN and OUT instructions.

It ranges from PLO to PL3

It determines current task on can execute I/O instructions.

4) Requested privilege level (RPL)

RPL of original supplier of selector.
determined by least two significant bits of selector.

Indicates privilege level requested by SIW when accessing segment.

5) Effective privilege level (EPL)

Least priority of RPL and DPL.
Since smaller RPL values indicate greater privilege, EPL is numerical max of RPL and DPL. It is used by CPU to enforce protection rules when accessing data segments.

* Privileged Instructions

1) Control Register Access

ex. `MOV CRx, reg` and `MOV reg, CRx`

→ Access control registers like CR0, CR3

→ It is used for enabling paging, protection

→ Manage fundamental CPU operations.

→ It is privileged because modifying control registers can alter CPU's operating mode and memory management.

2) Descriptor Table instructions

define and manage system's memory segmentation and interrupt

Handling structures.

ex. LGDT, SGDT, LIDT, SIDT, LLDT, SLDT

LTRISTR → load & store task register

→ These instructions set up foundational s/w structures for memory protection & interrupt handling.

→ unauthorized modifications could compromise system stability and security.

3) I/O instructions

Facillate direct communication with H/w devices through I/O ports.

ex. IN / OUT : Read write from I/O port

INS / OUTS : string.

Execution of these instructions also depends on I/O p.l. sets in EFLAGS register. Only code running at PL equal to or higher than 10PL can execute these Instru.

4) Interrupt control instructions

(Interrupt control instructions):-

• Manage processor's response to hardware and software interrupts.

ex.) CLI : clear Interrupt Flag (disable interrupt)
STI : set IF (enable interrupt)
HLT : Halt processor until next external interrupt.

→ controlling interrupts is critical for coordinating system activities and ensuring timely response to events i.e. these are PL instructions.

5) Task and segment instructions

Handle task switching and manage segment selectors for multitasking environments.

ex. LTR with segment selector for TSS.

LLDT with ---

CALL/JMP to task gate

These instructions control task management & memory segmentation, which are central to OS ability to manage multiple processes securely.

* Protection Mechanism

→ Takes place in address translation.

→ prevents misuse and corruption of system programs and data.

→ Three checks performed by processor:

1) Limit check

2) Type check

3) privilege check

1. Limit check

→ It is a key part of segmentation based protection mechanism.

→ It ensures that program cannot access memory outside its assigned segment, thus providing memory safety & isolation.

- Descriptor of target segment has a limit field, which gives max size of segment. Segment size can vary from 1 byte to 4GB.
- Offset address which is desired to be accessed by an instruction, must be within this limit, only access is granted.
- If not, general protection fault (INTERRUPT 13) occurs.

For byte access (8 bit), offset address $\leq$ limit

For word (16 bit), $0.A \leq (\text{limit} - 1)$

double word (32 bit), $0.A \leq (\text{limit} - 3)$

- limit here will also depend on granularity bit in segment descriptor.

CRAFTERS

2. Type check

- part of segmentation mechanism.
- ensures memory accesses conform to type and attributes of segment being accessed.

purpose :-

- 1) prevent illegal operations on segment.
- 2) To enforce privilege levels and access rules.
- 3) maintain system integrity and isolation bet^w tasks and processes.

Read only access (R/W) = 0

Read / Write access (R/W) = 1

When processor is executing at supervisory level, all pages are both readable or writeable.

When processor is executing at user level, only pages that belongs to user level and are marked for R/W access are writeable.

Pages that belongs to supervisor level are neither readable nor writable from user level.

3. Privilege check

→ 80386 provides 4 levels of protection called PL.

→ following privilege levels are checked before access to data or code.

i) RPL, DPL, CPL, EPL, IOPL

$U/S = 0 \rightarrow$ supervisor level

$U/S = 1 \rightarrow$ user level

if $CPL = 3 \rightarrow$ microprocessor executing at user level

$CPL = 0, 1, 2 \rightarrow$ mp executing at supervisor level

When mp is operating at supervisor level all pages are addressable but

DATE / / PAGE NO.

when its executing at user level only pages belonging to user level are addressable.

* Page level protection

It is a memory protection mechanism where different permissions (read, write, execute) are assigned to each memory page.

It prevents unauthorized access to memory, helping avoid issues like buffer overflows or code execution from non code areas.

Importance : security, stability, multitasking
example :-

Page Table Entry has
Flags like

- 1) R/W Read write access
- 2) U/S user supervisor -1-

Note :- explain Type checking & privilege check

* Aspects of protection mechanism

1) Type check

2) Limit check

3) Restriction of addressable domain

→ Limits memory area that a process can access.

→ ensures isolation has between processes and OS.

→ ex.) Each process has its own address space and cannot access other's memory.

It is implemented through segmentation and paging.

each segment has segment descriptor and access rights.

Paging allows further granularity with page level access control.

4) Restriction of procedure Entry Points

→ implemented through call gate

→ call gates in GDT define controlled entry points for procedure calls across PIs.

→ This ensures secure transitions from user to kernel mode.

→ It ensures all transitions go through well defined entry points that perform security checks.

5) Restriction of instruction

→ 80386 has PI instructions that can be only executed at Ring 0, or PLO.

→ user mode (PL3) attempting to run these instructions causes general protection fault.

→ This ensures separation of concerns and security by providing Ring Base protection.

* Combining Page and Segment Protection

80386 mp uses two level memory management system

- 1) segmentation
- 2) paging

Both level perform protection checks giving robust system.

Together they provide:

- multi level security
- process isolation
- fine grained memory control
- multitasking & VM

process:

When paging is enabled, first segment level protection is evaluated.

If everything is good then only page level protection is evaluated.

If processor detects violation of protection at any stage, requested operation can't proceed and an exception occurs.

	Segment level protection	Page level protection
Base Address size	Any any size	4kB aligned 4kB defined
Present (P)	Yes	Yes
A bit status	Yes	Yes
PL	4	2
Priority bit status	Not present	present
Write protection	Yes	Yes

1) segmentation stage

protection checks

- 1) is offset within segment limit
- 2) is P2 correct
- 3) is operation allowed

if any fail $\rightarrow$ General protection fault (#GP)

2) paging stage

CRAFTERS 

protection check

- 1) is page present in memory
- 2) is it writable
- 3) is user mode allowed to access it.

if any check fail $\rightarrow$ Page fault (#PF)

Why combine both

- Seq. 1) Logical structure, access rights, P2
- paging 2) VM, per page protection, memory isolation

* Confirming code segment

It allows to call lower PL code to call a higher PL code.

confirming flag in code segment descriptor should be set (1) to indicate that segment is a confirming code segment.

use: when certain privileged operations or system services need to be accessed by user level program; without switching entire execution context to higher PL.

* CALL gate

80386 uses gate descriptor provide protection for control transfer betⁿ executable segments at different PL's.

Gate descriptor components:

- 1) selector: identifies segment to execute code.
- 2) Gate type: identifies type of gate (call, Task, Interrupt)
- 3) offset: specifies offset with targeted code segment.
- 4) PL: specifies PL to execute code segment
- 5) start segment & pointer: specify s.s and s.p to be loaded when transfer occurs.

- call gate has two primary functions
- 1) To define entry point of procedure
 - 2) To specify PL of entry point

→ used by call & jump instructions in same manner as code segment descriptors.

Process:

When program encounters call gate instruction, it triggers transfer of control to specified code segment.

→ privilege level check

mp checks if PL of calling program or task is sufficient to execute code segment specified in call gate.

→ Code segment loading

If PL is appropriate processor loads new code segment specified in call gate into code segment register (CS).

→ Stack setup

mp set up stack segment (SS) & segment pointer (SP) values specified in call gate.

→ control transfer

processor transfers control to offset specified in call gate.



CRAFTERS 

CRAFTING A FUTURE

crafters.official.code@gmail.com



Subscribe
Today!



CRAFTERS 

@Crafters.official_code